

Preguntas de primera sesión

1. ¿Qué sucede exactamente en el servidor cuando un usuario ingresa a un dominio como *misitio.com* y solicita una página? **Hay una petición get para que se solicite el código html y el código lo haga.**
2. ¿Por qué se utiliza el formato JSON en lugar de texto plano cuando queremos enviar datos al servidor mediante una petición POST? **Por simplicidad, así no necesitamos una ia que tiene que interpretar el texto plano, con JSON ya viene estructurado para que sea fácil de leer.**
3. Imagina que te equivocaste en una sola letra del título de un post. ¿Qué verbo HTTP deberías usar para corregirlo y por qué no usarías el otro verbo de actualización?

Usaría patch ya que solo actualiza una parte del post mientras que put actualiza TODO EL POST.

4. En la arquitectura clásica MVC (Modelo-Vista-Controlador), ¿por qué es tan difícil reutilizar el mismo servidor para crear una aplicación móvil nativa (como una app para iPhone)? **Porque no podrían funcionar al mismo tiempo ya que una APP no puede ejecutar HTML de forma nativa.**
5. ¿Qué significa que en la arquitectura basada en APIs (REST) el backend se vuelva "agnóstico"? **Que no sea dependiente de una plataforma o un servidor, o sea que este separado en partes diferentes para que no se sature.**
6. Si una aplicación basada en APIs (Monolítica) crece y un solo módulo (como el de "Reportes PDF") satura el servidor, ¿qué problema enfrentamos al intentar usar el "Crecimiento Horizontal"? **Que al clonar los servidores pues los servidores se seguían saturando ya que también tienen el módulo de reportes, lo que no solucionaría el problema solo lo dispersaría.**
7. ¿Cómo solucionan los microservicios el problema de la "reacción en cadena" cuando falla un módulo pesado dentro de un sistema? **Que no son dependientes, si uno no jala los demás siguen jalando.**
8. Si los microservicios son tan buenos, ¿por qué no siempre es una opción para la arquitectura de un sistema? **Porque pueden ser complejos y difíciles de trabajar si el sistema no lo requiere.**
9. ¿Por qué es una mala práctica crear una "Lambda Monolítica" (meter toda la app en una sola Lambda) y cuál es la regla de oro que se recomienda en su lugar? **Porque es inseguro, ineficiente en recursos y puede perderse información por el tiempo de duración de la lambda, la opción es separar en lambdas con diferentes responsabilidades, muchos lambdas en vez de solo uno.**
10. En la charla se usó la analogía de "ir a la tiendita de la esquina". ¿Qué arquitectura representa el ir caminando, cuál representa la bicicleta y cuál representa el tráiler/motocicleta? **Caminar = monolito del mvc. Bici = APIs rest. trailer/moto = microservicios con lambda.**